The RAG cookbook

by Nuclia

May 2025

nuclia

The RAG cookbook

The rapid evolution of artificial intelligence (AI) has transformed the way we interact with information, creating new possibilities for more accurate, dynamic, and context-aware systems. One of the most promising advancements in this field is Retrieval-Augmented Generation (RAG), a cutting-edge approach that combines the power of large language models (LLMs) with a data retrieval system. This hybrid model enables more informed, flexible, and context-rich responses by integrating real-time access to controlled knowledge sources while generating human-like text.

RAG bridges the gap between traditional AI models, which rely solely on pre-existing knowledge stored within the model itself, and search-based systems that pull data directly from a database. By enhancing generation tasks with retrieval mechanisms, RAG enables more robust and contextually accurate outputs, addressing some of the core limitations of standalone generative models. This fusion opens the door to a wide range of applications, from improved conversational agents to advanced document summarization, personalized content creation, and even real-time decision support systems.

In this white paper, we will address the challenges and considerations in configuring RAG systems, providing insights into best practices.

Note: all the techniques detailed in this white paper are available in the Nuclia platform, you will find at the end of this document a list of useful options to configure your Nuclia RAG pipeline.

Index

Basic principles of RAG **2**
Evaluating the quality of the RAG pipeline **4**
Enriching the context **6**
Neighbouring text blocks or full documents 6
Metadata 6
Data augmentation 7
Advanced strategies for RAG **8**
Filtering on titles 8
Rephrasing 8
Automatic filtering 9
Boosting specific sources 9
Multi-steps queries 9
Injecting caveats 10
Annexes: Useful Nuclia options **11**
Search options 11
RAG & Generation options 11
Results options 12

Basic principles of RAG

RAG is a two-step process that combines a retrieval system with a generative model.

The first step involves retrieving relevant information from a knowledge source based on the user's query. This retrieval process is typically performed using a semantic search algorithm that matches the query against the available data and returns the most relevant text blocks. It can also be using results from a keyword search.

The second step is the generation phase, where a large language model (LLM) processes the retrieved information along with the user's query to generate a coherent and contextually relevant response. The LLM leverages the retrieved context to provide more informed and accurate answers. The user query and the retrieved context are concatenated in a prompt that will explicitly mention that the LLM is expected to use only the retrieved context to generate the answer. That way we make sure the LLM does not rely on its pre-existing knowledge.

These two steps can be tuned regarding different parameters, such as:

- The **amount of context retrieved**: the more text blocks are retrieved, the more information the LLM has to generate the answer. However, too much context can lead to irrelevant or noisy information. So you need to find the right balance, and it usually depends on the type of information you are dealing with: in some cases, most questions have a very specific answer that can be found in a single text block, while in other cases, the answer requires a broader context.

- The **ranking** of the retrieved text blocks: given you will restrict to a limited amount of text blocks, you need to rank them to make sure the most relevant ones are used by the LLM. Semantic results and keyword results come with different scores which cannot be compared directly. A good practice is to use a re-ranking that evaluates the relevance of the text blocks according to the user query.

- The **type of LLM** used: it is mostly a cost/benefit analysis. The more powerful the LLM, the more accurate the answers, but the more expensive the computation. You need to find the right balance between the quality of the answers and the cost of the computation.

- The **prompt**: the prompt is the text that will be used by the LLM to generate the answer. It should be carefully crafted to guide the LLM in the right direction. It should be clear and concise, and it should explicitly mention that the LLM should use only the retrieved context to generate the answer.

Evaluating the quality of the RAG pipeline

Once you have played with these parameters and configured your RAG pipeline, you need to evaluate its quality. It is not a one-shot process, but an ongoing task that requires regular monitoring and fine-tuning. Indeed, the content of the Knowledge box can change as you are ingesting new resources, and questions that used to be answered correctly can become incorrect. Also, the questions asked by the users can evolve, and all of a sudden, they might ask about topics that are insufficiently covered by your resources.

As it is a complex and critical task, Nuclia has developed REMi (it stands for RAG Evaluation Metrics), an efficient open-source fine-tuned LLM that simplifies the assessment of RAG pipelines.

REMi considers the main inputs/outputs in the RAG pipeline, which are:

- Query: The user's question, which the model will try to answer
- Context: The information retrieved by the retrieval step, which aims to be relevant to the user's query.
- Answer: The response generated by the language model after receiving the query and context pieces.

Hence, REMi defines the following metrics to assess the quality of the RAG pipeline:

- Answer relevance: relevance of the generated answer to the user query.
- Context relevance: relevance of the retrieved context to the user query.
- Groundedness: degree to which the generated answer is grounded in the retrieved context.

By combining these metrics, REMi provides a comprehensive view of the quality of the RAG pipeline.

For example:

- If the **context relevance is high** but the **answer relevance and groundedness are low**, it means that the model is generating **evasive answers**. The semantic search successfully retrieves relevant context pieces, but the model fails to generate a relevant and grounded answer. You should try to use a different LLM.

- If the **answer relevance is high** but the **context relevance and groundedness are low**, it means that the model is generating **unverifiable answers**. The LLM generates a relevant answer, but not based on the information stored in your Knowledge Box. First, you should check if the information is missing in your Knowledge Box. If the information is present, you should try to change your search and RAG strategy parameters.

- If the **groundedness is high** but the **answer relevance and context relevance are low**, it means that the model is generating **unrelated answers**. The LLM generates an answer based on the context, but the answer is not relevant to the user query. This can happen if the wrong context pieces are retrieved, but the LLM still feels compelled to generate an answer based on the available information, disregarding the nuances of the query.

We will focus here on cases where the information is present in the Knowledge Box, but the retrieval step is not providing the relevant context to the LLM.

Enriching the context

As mentioned earlier, RAG is about retrieving the best text blocks to answer the question. And the way it finds the "best" text blocks is by using semantic search (plus keyword search optionally). It means it assumes that the best text blocks are the ones that are the most semantically similar to the user query and the most relevant to produce the answer.

It is usually a good assumption, but not always, because some critical information might be located around the semantic matches without being retrieved.

That's why it might be very useful to **enrich the context** by adding some extra contents to the initial retrieval.

Neighbouring text blocks or full documents

When the information is scattered across multiple text blocks, it might be useful to retrieve the neighbouring text blocks to the ones that are semantically similar to the user query.

For example, in a user support context, imagine a long introduction presenting a typical problem, followed by a list of solutions. It is very likely that the user will describe their problem (as they probably do not know about the solution), so the introduction will have a very good semantic score, while the text blocks describing the solution might not.

In this case, if you know your documents are structured this way, you could consider adding systematically the 2 or 3 text blocks following the ones that are semantically similar to the user query, making sure you provide the LLM with the information it needs to generate the answer.

Adding too many text blocks might produce noise and lead to irrelevant information, so you need to find the right balance.

But if your documents are small, on the contrary, you might even consider adding the full document to the context.

Metadata

Another way to enrich the context is to append metadata. The most common metadata is the title of the document. Let's imagine your documents are articles about famous scientists, and let's consider the following case:

- The user's question is "Who discovered that the Universe is mostly made of hydrogen?".
- The document containing the answer is titled "Cecilia Payne-Gaposchkin".
- One of its text blocks contains the following sentence: "She is the genius who discovered that the Universe is mostly made of hydrogen".

This text block will be retrieved, as it is semantically very close to the question. Yet, it does not contain the answer to the question. The title does, but it will not be retrieved (it does not mention the universe nor hydrogen).

In this case, you can add the title of the corresponding document of each retrieved text block to the context, and the LLM will be able to provide a good answer.

The title is not the only metadata you can use. You can also consider the author, the publication date, labels and tags, etc.

Data augmentation

Data augmentation is a technique used to automatically generate extra contents from the existing ones. A very common one is summarization: you can use a summarization model to generate a summary of each document at ingestion time, and add it to the context whenever a text block from this document is retrieved.

For example, if the user asks "What is the best hammer for laying fabric on furniture?", and the corresponding document is "Upholstery hammer", but the retrieved text does not describe the hammer, it just mentions it is good at laying fabric on furniture. By adding the title as described in the previous section, the LLM would answer "Upholstery hammer", but it would not be very informative. If you add the summary, the LLM would be able to generate a more detailed answer.

You can use data augmentation techniques to create different types of contents, like insights about a given topic (like a security warning highlighting the risks of a given practice). It can also produce JSON structured data, like a list of brands, a list of prices, etc., that can also be valuable to the LLM.

Knowledge graph

A Knowledge Graph (KG) is a structured representation of information that captures relationships between "entities". Entities can be people, places, concepts, or any other items of interest, generally the nature of these entities are specific to a given business domain. KGs are typically represented as a graph, where nodes represent entities and edges represent the relationships between them. This structure allows for efficient querying and reasoning about the data.

In the context of Retrieval-Augmented Generation, KGs can be used to enhance the retrieval process by providing additional context and relationships that can improve the quality of generated responses.

Compared to semantic search, graph search has a broader view, it can gather information spread in distinct locations. Typically, semantic search will be efficient when the answer is concentrated in a few blocks of text that can be identified as good semantic matches to the query. For example, if our corpus is composed of documents describing the economic activities of all the countries, you can ask "What are the main industries in Italy?" and the answer will be in a single document. In this case, semantic search will be perfectly efficient. On the other hand, if you ask "What are the main industries in Europe?", the answer will be spread across many documents, moreover it implies to first know which countries are in Europe. That's where a knowledge graph can help. The KG can be used to identify the countries in Europe and then retrieve the relevant information about their industries from the documents.

It also performs much better for questions requiring exhaustivity, like "List all the products sold by company XYZ", or combining logical filtering, like "List all the products sold by company XYZ that are also sold by company ABC".

Building a KG is more complex than semantic indexing, as it requires first identifying and properly describing the kind of entities and relationships that are relevant to the business domain. This is typically done by a domain expert, who will define the schema of the KG. Once the schema is defined, the entities and relationships can be extracted from the documents using Named Entity Recognition (NER). The extracted entities and

relationships are then stored in a graph database, which allows for efficient querying and reasoning about the data.

When using a KG in RAG, you can combine the strengths of both semantic search and graph search. The KG can be used to enhance the retrieval process by providing additional context and relationships, while the semantic search can be used to quickly identify relevant text blocks. This combination allows for more accurate and comprehensive responses to user queries.

Advanced strategies for RAG

Here is a list of interesting examples of advanced strategies we have developed with our clients. Each of them are specific to a given use case, but they can be used as a source of inspiration to design your own strategy.

Filtering on titles

You could be in a context where a given topic is covered by both very general documents (discussing many different topics) and very specific ones (focusing on this given topic).

If you want to make sure the LLM uses the specific documents, and if you know that the title of the document is a good indicator of the relevance of the content, you can use it to filter the retrieved text blocks.

Usually in that case, it is more efficient to perform a keyword search on titles only, and then retrieve the text blocks only from the documents that have been found, using this time both semantic and keyword search.

Rephrasing

Rephrasing the user query is a common technique to improve the quality of the semantic search results (it is usually not recommended for keyword search as it can produce unexpected results).

Indeed the user query is not always well formulated, and the semantic search might not find the best text blocks because of that. The regular rephrasing is just about optimizing the query to make sure the best text blocks are retrieved.

But you can also use a specific prompt to make rephrasing more specific.

A great example is about multilingual context: the documents are all in english, but you want to let users ask questions in their own language. Usually, you would use a multilingual embedding model to index your content, and it should work well. But in some cases, an English embedding model happens to be more efficient. So you would prefer to use the english embeddings, but you still want to let users ask questions in their own

language. In that case, use a rephrasing prompt that rephrases the user query in english, and then use the english embeddings to retrieve the context.

Automatic filtering

Semantic search might make very strong matches on content that are explicitly not relevant to the user query, like "What was the biggest success of The Beatles in 2003?" and you have a content saying that "Yellow Submarine" was a huge success without mentioning it was in 1968. It will probably be a good match.

To avoid it, you need to extract from the user query anything that could be considered as a filter, and then use it to narrow down the semantic search results to the contents which metadata match the filters.

Boosting specific sources

Sometimes, not all the sources are equally reliable, and you want to make sure the LLM uses the most reliable ones to generate the answer.

Unfortunately, the semantic search might find the best text blocks in the less reliable sources, because they are the most semantically similar to the user query, and the valuable information is then drowned in the noise or even not surfaced at all.

To make sure to retrieve the text blocks from the most reliable sources, you can duplicate your query:

- run it on the reliable sources only, using the appropriate filtering,
- run it on the rest, with a reverse filter.

And then merge the two results set by applying a higher weight to the valuable sources.

That way, the reliable sources will obtain a better ranking, and the LLM will use them to generate the answer.

When running the 2 queries, besides the filtering to target a specific kind of source, you can also consider a specific rephrasing prompt to fit the language of the sources. For example, if the reliable sources are legal documents, you can use a legal vocabulary in the prompt, while you can keep a more general vocabulary for the rest.

Multi-steps queries

Sometimes boosting specific sources is not enough, what you really want is to get an answer from a specific part of your information corpus if this answer exists there, and only if it does not, you want to use the rest of the corpus.

For example, if your corpus contains contracts, amendments and meeting minutes, when answering a question you want to first consider the amendments, then the contracts, and finally the meeting minutes.

To do so, you need to run a multi-steps query:

- run the query on the amendments only,
- if it does not return any valuable result, run it on the contracts only,
- if it does not return any valuable result, run it on the meeting minutes.

And based on the collected results, you can let the LLM generate the answer.

It might also be combined with automatic filtering, so when the user asks "What do the amendements say about the new policy?", you run the first step query only.

Injecting caveats

Language models might not be always able to distinguish domain-specific terms that might be lexically similar but have different meanings. That's why fine-tuned LLMs can be very useful. But what if you have to use a generic LLM?

Let's imagine you are in a financial context, the LLM might be confused between "qualitative easing" and "quantitative easing". To help the LLM, you can inject a caveat in the prompt, providing the definition of both terms and the important differences between them, and insisting on the fact that the LLM should be careful about it when generating the answer.

But "qualitative easing" and "quantitative easing" won't probably be the only case of confusion in the financial domain lingo. So adding all these caveats in the prompt might be a bit heavy.

Instead, you can create a new resource, a "caveats" resource, that will contain all the caveats. And then when a user asks a question, you can run a normal query with their question plus a keyword search on the caveats resource, and then add the retrieved text blocks to the context.

Note: a keyword search is recommended here because we can assume the user question is not about the difference between "qualitative easing" and "quantitative easing", so the corresponding caveat text block is not likely to be found in the semantic search results.

Annexes: Useful Nuclia options

The strategies described in this white paper can be implemented in the Nuclia platform using the following options.

You can get the details of these options in the Nuclia documentation.

Search options

- features (semantic and/or keyword): to choose the type of search to perform.
- filters and filter_expression: to narrow down the search to specific sources according to their metadata.
- keyword_filters: to narrow down the search to sources containing some given keyword.
- autofilter: to automatically filter the search results based on the user query.
- top_k: to limit the number of text blocks retrieved.
- reranker: to re-rank the retrieved text blocks according to the user query.
- rephrase: to rephrase the user query for better semantic search results.
- prompt.rephrase: to provide a custom rephrasing prompt that will be used to rephrase the user query.

RAG & Generation options

- generative_model: to choose the LLM to use.
- prompt.user: to provide a custom user prompt that will be used by the LLM to generate the answer.
- prompt.system: to provide a custom system prompt that will be used by the LLM to generate the answer.
- rag_strategies:
    - full_resource: to retrieve the full document when a text block is retrieved.
    - field_extension: to retrieve some additional fields from the document when a text block is retrieved.
    - hierarchy: to retrieve the title and the summary when a text block is retrieved.
    - neighbouring_paragraphs: to retrieve the neighbouring text blocks when a text block is retrieved.

    - metadata_extension: to retrieve some metadata when a text block is retrieved.
    - prequeries: to run extra queries before the main one to either filter the sources, or add more context from different parts of the corpus.
    - graph_beta: to retrieve relations from the knowledge graph.
- rag_images_strategies: to pass either images contained in the retrieved text blocks to the LLM, or a full screenshot of the page containing the text block.

Results options

- answer_json_schema: to produce a JSON structured answer, computing specific attributes based on the context.
- citations: to provide the sources of the retrieved text blocks.
- citation_threshold: to increase or decrease the accuracy of the citations.

nuclia.com